Relatório de Reengenharia

Reestruturação e engenharia direta da classe x_Mgr_F

1. Modelo local utilizado

Qwen 2.5 Coder 7B, rodando no Ollama com `ollama run qwen2.5-coder`. Abrimos uma sessão nova para essa parte.

2. Prompt original

Esse foi o prompt que mandamos para o modelo. Colocamos nele o dicionário de dados da aula de engenharia reversa. A diferença é que dessa vez não chamamos `st` de "Status", porque na aula passada vimos que isso tinha sido invenção da IA. Então pedimos para o modelo não inventar significado nenhum para ela.

```
Você vai atuar como um Arquiteto de Software Sênior especialista em Java, Clean Code e SOLID.

Sua tarefa é fazer a reengenharia do código legado abaixo. O comportamento funcional deve ser
preservado exatamente, mas a estrutura deve ser modernizada.

CÓDIGO LEGADO:
public class x_Mgr_F {
    public double calc(int tp, double[] vls, boolean st) {
        double r = 0;
        for (int i = 0; i < vls.length; i++) {
            r += vls[i];
        }
        if (tp == 1) {
            r = r - (r * 0.12);
            if (st) {
                r += 15.50;
            }
        } else if (tp == 2) {
            if (r > 500) {
                r = r - (r * 0.05);
            }
        }
        System.out.println("LOG: " + r);
        return r;
    }
}

DICIONÁRIO DE DADOS (validado pela equipe na engenharia reversa):
- x_Mgr_F: classe que calcula o valor final de um atendimento da clínica. Novo nome:
  CalculadoraDeAtendimento.
- calc: método que calcula o valor final do atendimento. Novo nome: calcularValorFinal.
- tp: tipo de consulta. 1 = Convênio, 2 = Particular. Qualquer outro valor: nenhuma regra é
  aplicada. Novo nome: tipoConsulta (use um enum TipoConsulta).
- vls: valores dos procedimentos realizados no atendimento. Novo nome: procedimentosRealizados.
- st: flag booleana que, somente no Convênio, soma uma taxa adicional fixa de 15,50 DEPOIS do
  desconto. O significado de negócio não está documentado, então não invente um. Novo nome:
  cobrarTaxaAdicional.
- r: valor total acumulado e retornado. Use nomes como subtotal e valorFinal.

REGRAS DE NEGÓCIO (devem ser preservadas exatamente):
1. Subtotal = soma de todos os procedimentos realizados.
2. Convênio: subtrair 12% do subtotal. Se cobrarTaxaAdicional for verdadeiro, somar 15,50 depois
   do desconto.
3. Particular: subtrair 5% somente se o subtotal for estritamente maior que 500. Com 500 exatos,
   não há desconto.
4. Outro tipo de consulta: devolver o subtotal sem alteração.

REQUISITOS DE ARQUITETURA:
```

```
1. Use Java 17.
2. Aplique Clean Code: nomes claros em português, sem abreviações e sem números mágicos (use
   constantes com nome).
3. Aplique o Princípio da Responsabilidade Única (SRP): separe a soma dos procedimentos da
   aplicação das regras de desconto.
4. Aplique o padrão Strategy: crie uma interface RegraDeCobranca com uma implementação para
   Convênio, uma para Particular e uma sem regra (para tipos não identificados).
5. Aplique o padrão Factory: crie uma classe que devolve a RegraDeCobranca correta a partir do
   TipoConsulta.
6. Não use System.out.println dentro das classes de regra de negócio.
7. Use double, como no legado. Some os procedimentos com um laço simples, na mesma ordem do
   legado. Não use streams nem BigDecimal, para que o resultado seja idêntico ao do sistema antigo.
8. Não adicione regras, validações ou exceções que não existam no código legado.

FORMATO DA RESPOSTA:
- Entregue cada classe em um bloco de código separado, com um comentário na primeira linha
  indicando o nome do arquivo (ex.: // Arquivo: TipoConsulta.java).
- Depois, em no máximo 5 linhas, explique como o novo design aplica SRP, Strategy e Factory.
```

3. O que a IA entregou

Comparado com a aula passada, o resultado foi bem melhor. O modelo criou o enum, a interface `RegraDeCobranca`, uma classe de regra para cada tipo de consulta e uma fábrica para escolher a regra. Tirou o `System.out.println` do cálculo, manteve o "maior que 500" e somou a taxa de 15,50 depois do desconto, do jeito que o legado faz.

Mas quando colamos o código na IDE, apareceram alguns problemas. Teve coisa de Clean Code e teve coisa que nem deixava a gente testar. O código do jeito que a IA mandou está no anexo.

4. O que tivemos que arrumar

Nome do enum errado. A IA escreveu `PARCULAR` no lugar de `PARTICULAR`, e repetiu o erro na fábrica. Compila normal, então se ninguém lesse passaria batido. Corrigimos nos dois lugares.

Não tinha como usar o código 1 ou 2. O sistema antigo recebe o tipo como número, mas o enum que a IA criou não tinha nenhuma forma de converter isso. Sem isso a gente não conseguia nem montar o paciente do tipo 1 que a atividade pede. Criamos o `TipoConsulta.doCodigo(int)`.

Números mágicos. Mesmo com o prompt pedindo constantes, o 0.12, o 15.50, o 500 e o 0.05 continuaram soltos no meio do código. Passamos todos para constantes com nome, tipo `PERCENTUAL_DESCONTO` e `LIMITE_PARA_DESCONTO`.

A soma continuou dentro da calculadora. A gente pediu para separar a soma dos procedimentos das regras de desconto, e a IA separou só as regras. O laço da soma ficou dentro da `CalculadoraDeAtendimento`. Tiramos para uma classe própria, a `SomadorDeProcedimentos`, que a calculadora recebe no construtor.

Fábrica. Coisa menor: a fábrica só tem método estático, então deixamos a classe `final` com construtor privado e usamos o switch novo do Java 17.

Também organizamos as classes em pacotes (`atendimento`, `cobranca` e `legado`) e montamos o projeto em Maven para rodar os testes com JUnit 5.

5. Testando se continua funcionando igual

A ideia da reengenharia é mudar a estrutura sem mudar o resultado, então precisávamos provar isso. Deixamos a classe `x_Mgr_F` original no projeto, sem mexer em nada, e fizemos uma `Main` que roda o mesmo caso nos dois códigos e compara. Se o valor do código novo for diferente do legado, ou diferente da conta feita na mão, aparece FALHOU.

Além do caso que a atividade pede (tipo 1, procedimentos [200.0, 100.0] e taxa ativa), aproveitamos os casos estranhos que tínhamos achado na aula de engenharia reversa:

Cenário	Entrada	Esperado
Convênio com taxa (atividade)	tp=1, [200.0, 100.0], true	300 - 12% = 264 + 15,50 = 279,50
Convênio sem taxa	tp=1, [200.0, 100.0], false	264,00
Particular acima de 500	tp=2, [300.0, 300.0], false	570,00
Particular com 500 exatos	tp=2, [250.0, 250.0], false	500,00
Tipo que não existe	tp=3, [200.0, 100.0], false	300,00
Convênio sem procedimentos	tp=1, [], true	15,50

Saída da Main (os LOG são da classe antiga, que ainda tem o println):

```
LOG: 279.5
[OK] Convênio com taxa (cenário da atividade) | novo: 279.50 | legado: 279.50 | esperado: 279.50
LOG: 264.0
[OK] Convênio sem taxa | novo: 264.00 | legado: 264.00 | esperado: 264.00
LOG: 570.0
[OK] Particular acima de 500 | novo: 570.00 | legado: 570.00 | esperado: 570.00
LOG: 500.0
[OK] Particular com 500 exatos | novo: 500.00 | legado: 500.00 | esperado: 500.00
LOG: 300.0
[OK] Tipo não identificado | novo: 300.00 | legado: 300.00 | esperado: 300.00
LOG: 15.5
[OK] Convênio sem procedimentos | novo: 15.50 | legado: 15.50 | esperado: 15.50
```

Passou tudo. O caso da atividade deu 279,50 nos dois códigos. Além da Main, deixamos os mesmos casos em testes JUnit, junto com alguns testes de cada regra separada.

Uma coisa que percebemos: o último caso cobra 15,50 de um atendimento sem nenhum procedimento. Isso parece bug do sistema antigo, mas mantivemos igual, porque o objetivo aqui era não mudar o comportamento. Corrigir isso seria uma decisão para conversar com a clínica.

6. Comparativo arquitetural

No código antigo, um método só fazia tudo: somava os valores, via o tipo de consulta num monte de IF, aplicava desconto, somava taxa e ainda imprimia no console. Pelo SRP, uma classe deveria ter um motivo só para mudar, e essa tinha vários. Se o convênio mudasse a porcentagem, se aparecesse um tipo novo de consulta ou se o log tivesse que ir para outro lugar, a gente ia ter que mexer sempre no mesmo método e torcer para não quebrar o resto.

No código novo cada parte ficou no seu canto. O somador soma, cada regra cuida de um tipo de consulta, a fábrica escolhe a regra e a calculadora só junta tudo. Com o Strategy, se amanhã entrar um convênio com outra regra, é só criar uma classe nova que implementa a interface, sem mexer no que já funciona (Aberto/Fechado). Como regra de convênio costuma mudar bastante, isso faz diferença.

Pensando nas refatorações do Fowler, usamos basicamente: *Rename Variable* (tp, vls, st e r viraram nomes que dá para entender), *Replace Magic Number with Symbolic Constant* (os percentuais e valores fixos), *Extract Class* (a soma saiu para outra classe) e *Replace Conditional with Polymorphism* (os IFs de tipo viraram as classes de regra). E o println saiu da regra de negócio, que não é lugar dele.

A longo prazo, o maior ganho é que dá para entender e testar. Quem pegar esse código não vai precisar fazer engenharia reversa como a gente fez, e cada regra pode ser testada sozinha. Sobre a IA: ajudou bastante a chegar no desenho, mas o código que ela entregou não estava pronto. Errou nome, ignorou parte do que pedimos e deixou o SRP pela metade. Sem a revisão do grupo, ia ser entregue com problema.

Anexo: código original gerado pela IA, antes dos ajustes

```
// Arquivo: TipoConsulta.java
public enum TipoConsulta {
    CONVENIO, PARCULAR, OUTRO
}

// Arquivo: RegraDeCobranca.java
public interface RegraDeCobranca {
    double aplicar(double subtotal, boolean cobrarTaxaAdicional);
}

// Arquivo: RegraDeCobrancaConvenio.java
public class RegraDeCobrancaConvenio implements RegraDeCobranca {
    @Override
    public double aplicar(double subtotal, boolean cobrarTaxaAdicional) {
        double valorFinal = subtotal - (subtotal * 0.12);
        if (cobrarTaxaAdicional) {
            valorFinal += 15.50;
        }
        return valorFinal;
    }
}

// Arquivo: RegraDeCobrancaParticular.java
public class RegraDeCobrancaParticular implements RegraDeCobranca {
    @Override
    public double aplicar(double subtotal, boolean cobrarTaxaAdicional) {
        if (subtotal > 500) {
            return subtotal - (subtotal * 0.05);
        }
        return subtotal;
    }
}

// Arquivo: RegraDeCobrancaOutro.java
public class RegraDeCobrancaOutro implements RegraDeCobranca {
    @Override
    public double aplicar(double subtotal, boolean cobrarTaxaAdicional) {
        return subtotal;
    }
}

// Arquivo: FabricaDeRegrasDeCobranca.java
public class FabricaDeRegrasDeCobranca {
    public static RegraDeCobranca criarRegraDeCobranca(TipoConsulta tipoConsulta) {
        switch (tipoConsulta) {
            case CONVENIO:
                return new RegraDeCobrancaConvenio();
            case PARCULAR:
                return new RegraDeCobrancaParticular();
            default:
                return new RegraDeCobrancaOutro();
        }
    }
}

// Arquivo: CalculadoraDeAtendimento.java
public class CalculadoraDeAtendimento {
    public double calcularValorFinal(TipoConsulta tipoConsulta, double[] procedimentosRealizados,
boolean cobrarTaxaAdicional) {
        double subtotal = 0;
        for (double valor : procedimentosRealizados) {
            subtotal += valor;
        }
        RegraDeCobranca regraDeCobranca = FabricaDeRegrasDeCobranca.criarRegraDeCobranca(tipoConsulta);
        return regraDeCobranca.aplicar(subtotal, cobrarTaxaAdicional);
    }
}
```